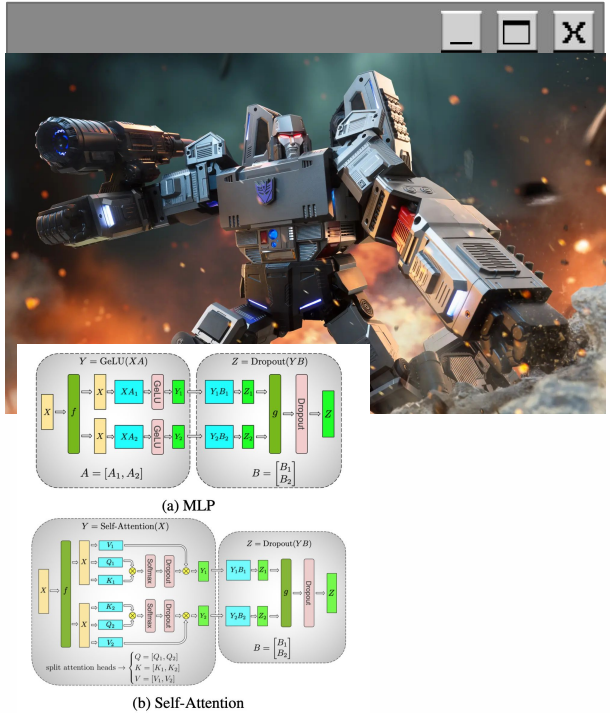
Mega Megatron



Kelly Xu, Kayla Liang, Janet Meng, Ina Ding

Oh no!

The model you're using when you talk to ChatGPT has hundreds of billions of parameters. A single H100 GPU — the best available — has 80GB of memory. GPT-3 alone needs roughly 350GB just to store its weights.

So how do such large transformer models get trained at all?

Not just a ML problem, it’s a *systems* problem…

Background

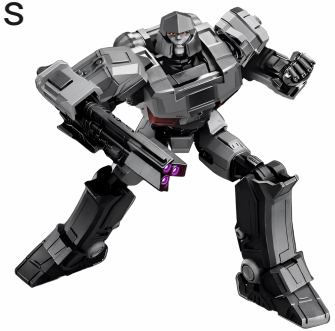
- Training large transformer models have advanced the capabilities of NLP applications
- However, training them is very data-intense and often limited by memory constraints
- Megatron-LM explores a parallel approach that enables training transformer models with billions of parameters

What is a transformer?

Compared to RNNs, transformers process tokens in parallel, as opposed to sequentially utilizing “attention mechanisms”

With self-attention operation, each token becomes a **context-aware** representation of the sequence

Transformers use **multi-head attention,** with each head learning in parallel



I didn’t know megatron was a real transformer…

What we know so far

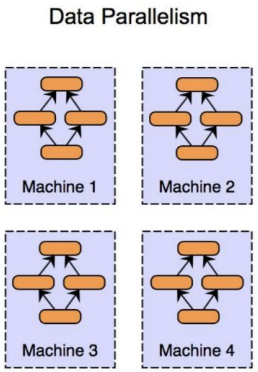
Bigger language models → better NLP performance.

How do we **scale model size** for transformers?

Parallelism: spread it across multiple GPUs. But *how* you spread it matters a lot!

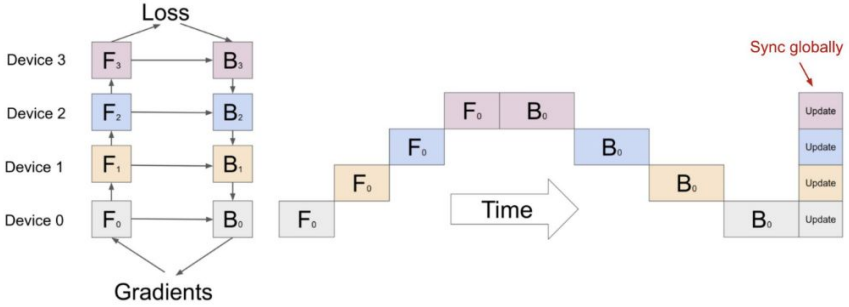
Three Forms of Parallelism

1. Data Parallelism (replicated workers)
   a. Copy the full model onto each GPU, but give each one a different mini-batch of data
   b. After each fwd & backward pass, workers sync gradients & model weights
   c. Pros: simple communication
   d. Cons: the entire model still needs to fit on 1 GPU (transformer model size is limited)



Three Forms of Parallelism

2. Model Parallelism (layer sharding)
   a. Split the model's *layers* across GPUs (GPU 1 runs layers 1-8, GPU 2 runs layers 9-16, etc).
   b. Pros: can train larger models now!
   c. Cons: GPUs sit idle waiting for the previous stage to finish → "pipeline bubble"

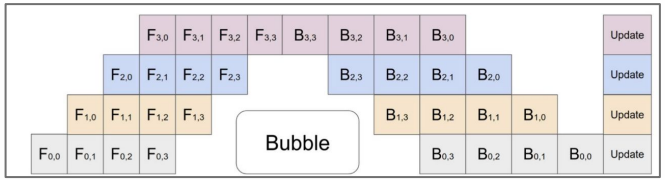


Three Forms of Parallelism

2.5 Pipeline Parallelism (layer sharding + microbatches of data)

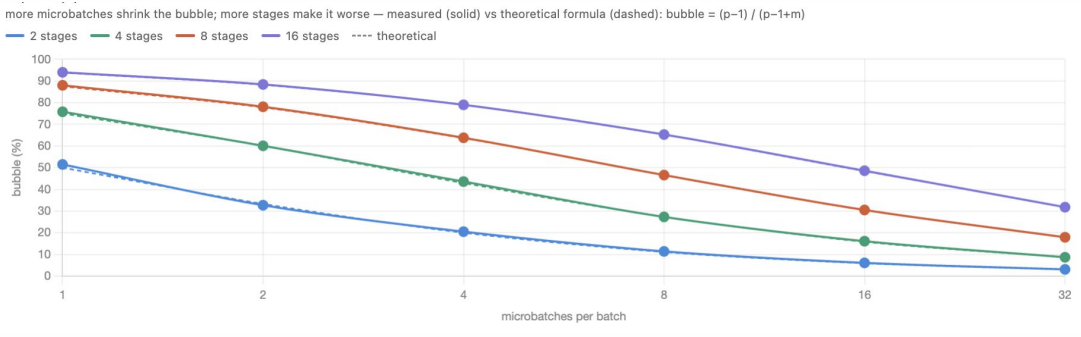
a. Split the model's layers across GPUs (GPU 1 runs layers 1-8, GPU 2 runs layers 9-16, etc).
b. Divide the mini-batch into micro-batches and process them sequentially → all devices are working for most of the time now
c. Pros: can train larger models now!
d. Cons: 1) bubble/idle time, 2) if layers are too big… still can't fit on one device

Theo bubble time = (p-1)/(p-1+m)
p = num pipeline stages/devices
m = num microbatches
If m>>p → bubble goes to 0



Bubble experiments

Pipeline bubble fraction deps on # microbatches and # stages



Bubble heatmap

darker = worse bubble; sweet spot is bottom-right (many microbatches, few stages)

stages ↓ / mb →	1	2	4	8	16	32
2	52%	33%	21%	11%	6%	3%
4	76%	60%	44%	27%	16%	9%
8	88%	78%	64%	47%	31%	18%
16	94%	88%	79%	65%	49%	32%

Three Forms of Parallelism

3. Tensor Parallelism
   - Split *individual layers themselves* across GPUs
     - Via weight matrices

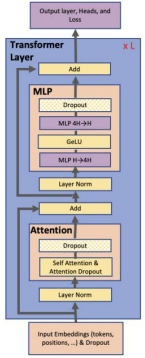
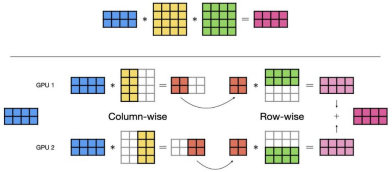


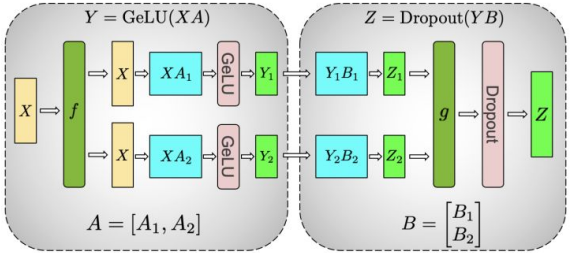
Figure 2. Transformer Architecture. Purple blocks correspond to fully connected layers. Each blue block represents a single transformer layer that is replicated N times.

Transformers: 2 main subcomponents
- Multi-head attention block
- Feed forward block

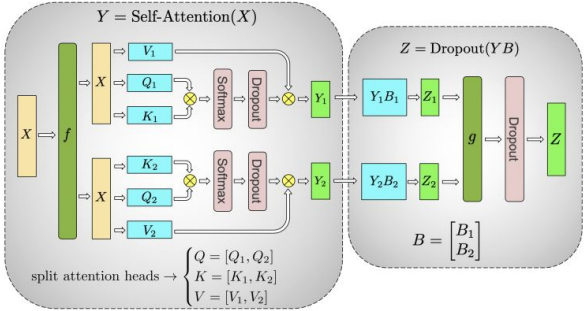
Multi-Head Attention:
- Naturally split (each attn head is indpt) → assign diff heads to diff GPUs

Feed forward block:
- 2 matrix multiplications





(a) MLP



(b) Self-Attention

Figure 3. Blocks of Transformer with Model Parallelism. f and g are conjugate. f is an identity operator in the forward pass and all reduce in the backward pass while g is an all reduce in the forward pass and identity in the backward pass.

Key Takeaways

"Keep GPUs compute-bound, not communication-bound"

- compute → communicate → compute → communicate ❌😴
- Compute a lot → communicate once ✅😴

Megatron-LM:

- Only requires 4 all-reduce communications per transformer layer

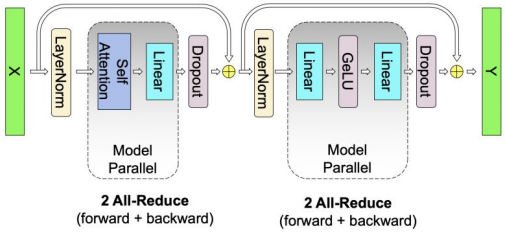


Figure 4. Communication operations in a transformer layer. There are 4 total communication operations in the forward and backward pass of a single model parallel transformer layer.

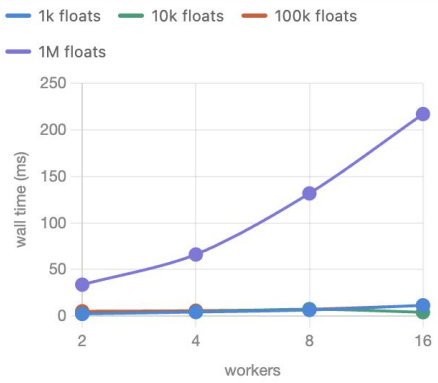
In practice…

Megatron caps tensor parallelism @ 8 GPUs

From a quick simulation of tensor parallelism's all-reduce operation:

- Latency gets worse w/ every GPU you add
- Communication overhead can dominate the benefit of splitting work across GPUs

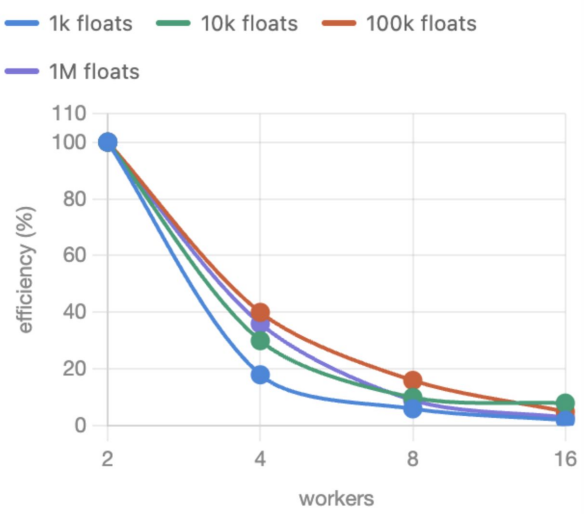
total gradient vector size (# of float64 elements)



Wall time = time it takes for every worker to end up w/ fully summed result

In practice…

- Efficiency drops sharply beyond 4-8 workers – communication overhead dominates compute :(
  - Smaller data size ~ lower efficiency
    - Underusing the hardware



Our Experiment

- Parallelizing the GEMM steps (X * A) in our Megatron's MLP block

- "Row-split"
  - Split weight matrix A along rows
  - Split input X along columns
  - Requires a synchronization point before GeLU

$$X = [X_1, X_2],\ A = \begin{bmatrix} A_1 \\ A_2 \end{bmatrix}$$

$$\mathbf{Y = GeLU(X_1A_1 + X_2A_2)}$$

- "Column-split"
  - Split A along columns
  - Leave X as normal

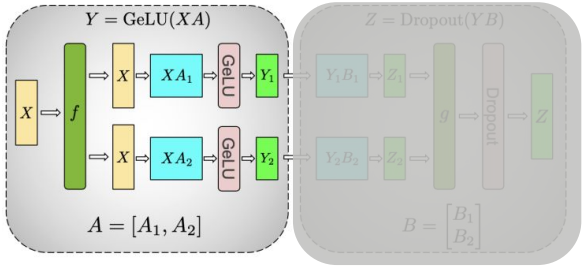
$$\mathbf{Y = [Y_1, Y_2] = [GeLU(XA_1), GeLU(XA_2)]}$$

https://github.com/liangkayla/cs390-final-project

Our Experiment

- Represent GPUs as goroutines

```go
// This is Figure 3a of the Megatron-LM paper.
func strategyPaperOptimal(X, A, B *mat.Dense, numGPU int) MLPResult {
    events := []CommunicationEvent{}
    computeTimes := make([]float64, numGPU)
    syncPts := 0

    // Partition weights
    As := SplitColumns(A, numGPU)
    Bs := SplitRows(B, numGPU)

    Yparts := make([]*mat.Dense, numGPU)
    Zparts := make([]*mat.Dense, numGPU)

    // — Stage 1: X·A_i (parallel)
    var wg sync.WaitGroup
    for i := 0; i < numGPU; i++ {
        wg.Add(1)

        go func(i int) {
            defer wg.Done()
            start := time.Now()
            Yi := MatMul(X, As[i])
            Yi = GeLU(Yi)
            elapsed := time.Since(start).Seconds()
            computeTimes[i] = elapsed
            Yparts[i] = Yi
        }(i)
    }
    wg.Wait()
```

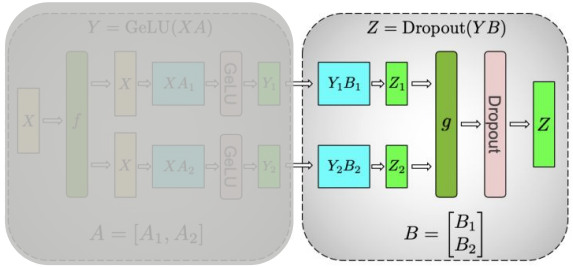


(a) MLP

Our Experiment

- Represent GPUs as goroutines

```go
// — Stage 2: Y_i · B_i (parallel)
for i := 0; i < numGPU; i++ {
    wg.Add(1)
    go func(i int) {
        defer wg.Done()

        start := time.Now()

        var C mat.Dense
        C.Mul(Yparts[i], Bs[i])

        elapsed := time.Since(start).Seconds()
        computeTimes[i] += elapsed

        Zparts[i] = &C
    }(i)
}
wg.Wait()

// — AllReduce
Z, ev := AllReduce(Zparts)
events = append(events, ev)
syncPts++
```



(a) MLP

Initial Results

```
MEGATRON-LM MLP SPLIT STRATEGY EXPERIMENT
batch=16  d_in=512  d_hidden=2048  d_out=512  gpus=8

[1] No Split (single GPU baseline)
    Full computation on one GPU, no communication

    Communication : none
    FLOPs            : 6.711e+07
    Compute time     : 0.003161 s
    Comm time        : 0.000000 s
    Sync points      : 0
    Total comm data  : 0.000 MB
    Relative error   : 0.00e+00  ✓ CORRECT

[2] Paper Optimal (col-split A, row-split B)
    Parallel GEMMs + 1 AllReduce

    Comm event #1 : AllReduce(gpus=8, bytes/gpu=131072)
    FLOPs            : 6.711e+07
    Compute time     : 0.002269 s
    Comm time        : 0.000052 s
    Sync points      : 1
    Total comm data  : 1.000 MB
    Relative error   : 1.60e-15  ✓ CORRECT
```

- For a small MLP block,1 GPU was better
- As simulated MLP blocks grow larger, the effects of distributed computation are more visible
- More GPUs = lower compute time but higher communication time

Distributed Systems Framing

- Intelligently distributing matrix operations across GPUs 🤝
  Intelligently sharding data across servers
- Communication bottleneck
  - Sync step must happen before downstream training steps
  - Where have we seen this before?
    - Cross-shard transactions (ie. 2PC)
    - Keeping replicas consistent (ie. RAFT)